NATIONAL UNIVERSITY OF SINGAPORE
School of Computing

Exploring New Information Visualization Techniques for VisuAlgo+

Rebuilding Interactive Algorithm Visualizations

CP4101 Final Year Project
Continuous Assessment Report

Truong Van Quoc Bao
A0276275U

Project ID: H160390
Supervisor: A/P Steven Halim
Main Evaluator: SIM Mong Cheng, Terence
Academic Year 2026/27, Semester 1
4 November 2026

Abstract. VisuAlgo+ is a collaborative project to rebuild VisuAlgo with a renewed technical foundation and a redesigned interface while preserving its educational purpose. My work focuses on the visualization and interaction layer: graph, tree and sorting visualizers, their editing and playback behaviour, and their integration into a common interface. The work began with standalone prototypes and progressed to a single application using a shared panel layout. This report explains the separation of input, layout and playback state; a shared panel-group layout (V10); algorithm extensions and their validation; and floating text, short explanations placed on the drawing. The current application contains 11 graph algorithms (counting the two DFS variants separately), seven tree workspaces and eight sorting algorithms. Its recorded validation on 1 November 2026 reports 3,273 passing unit tests and 151 passing browser cases. These results support functional progress. A formative user trial in Week 10 informed several UI/UX refinements, including floating text and adjustments to the legend and Status panel. Neither the tests nor the trial establishes improved learning outcomes or comparative usability. The next phase will strengthen interaction testing, evaluate representative learning tasks and investigate a limited integration with the separate WebAssembly engine.

1 Introduction and Objectives

1.1 Project background

VisuAlgo provides interactive visualizations of data structures and algorithms. Its original design brings different topics into a consistent environment and allows students to explore their own inputs [2]. These strengths are the starting point for VisuAlgo+.

In an early discussion, A/P Steven Halim described the existing Laravel/PHP, MySQL, D3 and jQuery stack. Some of these tools have become dated and no longer suit the planned future updates, so his aim for this version is to use the best currently available tools for both the interface and the execution engine. The rebuild still retains useful technologies, including D3 for selected layout and animation tasks.

The aim is a modern successor that preserves VisuAlgo's educational purpose and useful interactions while supporting future development. The current deliverable is an experimental application, not yet a full replacement.

1.2 Personal motivation and role

My motivation for joining VisuAlgo+ comes from my background in competitive programming and teaching. Participating in many programming competitions and achieving strong results has given me substantial experience with algorithms and data structures. Teaching students passionate about competitive programming has helped me explain these concepts from different perspectives and recognize where learners need clarification.

I hope to apply both experiences to VisuAlgo+'s user interface and user experience (UI/UX), helping learners follow why an algorithm makes a decision, how its state changes, and how those changes produce the result. This perspective guides which intermediate states I show, how I connect drawings with pseudocode, and where short explanations could clarify a step.

My initial responsibility is the visualization and interaction layer for graph algorithms, tree-based data structures and sorting algorithms. This includes developing the visualizers, supporting input editing and step-by-step playback, and integrating them into a consistent interface. Working with Kerk Tai Heng, a fellow FYP student, on the shared design and integration, I aim to turn my algorithmic knowledge and teaching experience into practical improvements that can later be evaluated with learners.

1.3 Objectives and current scope

The project has five working objectives:

1. Select suitable modern and widely adopted technologies for rebuilding the platform.
2. Preserve useful algorithm exploration, input editing and step-by-step explanation from VisuAlgo.
3. Provide a consistent interface across different algorithm and data-structure modules.
4. Establish clear boundaries for extending visualizers and integrating execution engines.
5. Validate the rebuilt functionality and identify what still requires technical and user evaluation.

Kerk, who has the same supervisor, works on the WebAssembly engine foundation, website design system and shared infrastructure. Section 3 details our contributions and collaboration.

The current application targets desktop and laptop use. It runs TypeScript algorithm implementations in the browser. The separate WebAssembly engine has not been connected to these visualizers. Mobile design, complete feature parity with VisuAlgo and measured learning improvement are outside the completed work reported here.

2 Background and Related Work

2.1 Algorithm visualization and educational use

Halim et al. [2] present the unified, interactive web-based environment for learning algorithms that later became VisuAlgo. Support for user-defined inputs and consistent interaction across topics is particularly relevant to this project. A reconstruction should preserve these properties even when the rendering technology and page layout change. Reusing familiar examples and algorithm operations also provides a useful reference when checking the rebuilt functionality.

However, a working animation is not sufficient evidence of learning benefit. Hundhausen, Douglas and Stasko's meta-study of 24 studies [3] found that how learners used algorithm visualizations mattered more than their graphical characteristics alone. This motivates a distinction between two questions in VisuAlgo+: whether the software behaves correctly, and whether students can use it to understand and explain an algorithm. The present tests primarily address the first question; the Week 10 trial (Section 6.1) gave formative input on the second, and a structured follow-up is planned.

OpenDSA combines visualizations with explanatory material, interactive exercises and proficiency assessment [4]. It illustrates how visualization can form one part of a wider learning activity. VisuAlgo+ currently concentrates on exploration: editing inputs, stepping through execution, following pseudocode and inspecting state. It does not yet provide an equivalent assessed learning sequence. Therefore, the current contribution should be judged as a visualization foundation rather than a complete instructional system.

2.2 Reusable platforms and implementation choices

Shaffer et al. [5] survey the algorithm-visualization field and discuss issues including availability, quality and topic coverage. Their survey motivates attention to maintainable implementations and reusable interaction patterns.

For the technical foundation, React Flow supplies an interactive node-and-edge canvas [9]. The project builds algorithm-specific state, edge identities and presentation rules above that canvas. D3 supplies narrower computational tools: force simulation for graph positions and a hierarchy layout for trees [1]. This combination separates the computation of positions from the ownership of application state. Sorting uses Canvas 2D because its main representation is an array of bars rather than an editable graph.

2.3 Position of this work

The project applies established algorithm, rendering and interface ideas to a practical rebuild. Its current value lies in connecting them coherently: a stable editable document, reproducible playback, shared page controls, and explanations tied to individual algorithm steps. Dockable panels and low-link algorithms are not claimed as new research techniques.

Three design questions guide the work. First, how can editing and playback share one canvas without accidentally invalidating or corrupting an execution? Second, how can different topics share a layout while preserving their own interactions? Third, how can a step remain understandable when it changes internal state but produces little visible motion? Sections 4.2–4.8 describe the implemented responses.

3 Development Approach and Responsibilities

3.1 From standalone prototypes to one application

The first stage, starting before the semester, explored interaction behaviour in standalone repositories. The first graph prototype has its first recorded commit on 28 July 2026; a second graph prototype followed in August with a clearer separation of document, layout and playback state. Sorting and tree prototypes explored different representations and operation models. This work provided a functional base before integration with the shared website.

During September and October, I adapted these visualizers to Kerk's design system. Kerk's wireframe of the visualization page drew nine candidate layouts, numbered V1–V9. After a user survey organized by Kerk, Kerk and I chose two of them, V2 (split rails) and V7 (docks), and I built the graph, tree and sorting visualizers in both layouts with comparable functionality (20–27 September). In Week 9, Kerk and I discussed a proposal to combine V2 and V7 into a tenth layout, V10, retaining selected strengths of each. I implemented V10, and Kerk contributed refinements to complete the version later used in the Week 10 trial. The survey served as formative input only; its responses are not analysed in this report.

On 18 October, Kerk consolidated the separate site and visualizer copies into one application and moved common code into a shared layer. Kerk and I then ran a separate formative user trial in Week 10 (Section 6.1). Subsequent work extended algorithms, added floating text and resolved defects found in review. Table 1 summarizes these stages.

3.2 Individual contribution and collaboration

My principal contributions are the standalone visualizer foundations, their integration, the core V10 layout model, algorithm and operation extensions, and the shared floating-text layer, together with most of their automated tests and design-decision records. Of the 48 numbered design choices in the V10 proposal, the first 26 were my decisions. Choices 27–48 (18–22 October) and the 18 October decision to keep only V10 were made by Kerk and me together and implemented by Kerk. Kerk also owns the original design system, engine foundation, infrastructure and application consolidation. These responsibilities overlap at integration boundaries [8].

I used Claude and Codex for implementation, debugging, test generation and review under my requirements and design choices. My acceptance workflow included reading changes, running the application and using tests or continuous integration (CI) results. In late-October review sessions, Codex identified findings, Claude reproduced findings, and I selected the fixes to pursue. AI also

Table 1: Main development stages.

Period	Main outcome
July–August	First graph prototype (first recorded commit 28 July, before the semester); second graph prototype with separated document, layout and playback state; standalone sorting prototype.
September	Tree workspaces; adaptation to Kerk's design system; V2 and V7 layouts built (20–27 Sept.); unmerged experiment on Kerk's engine.
29 Sept.–17 Oct.	Recursive DFS, Bellman–Ford and Prim; lazy segment tree and AVL deletion; playback refinements; V10 discussed (Week 9) and implemented.
18–31 Oct.	One-app consolidation (Kerk); LCA and heap operations; Week 10 user trial; topological sort, bipartite check, cut vertices and bridges; floating text; review fixes.
1–4 Nov.	Recorded validation (1 November) and report preparation.

assisted in organizing evidence and drafting this report. Such assistance is part of the development process; generated code and passing tests still require critical review, and do not establish correctness by themselves.

4 System Design and Implementation

4.1 Architecture of the current application

The current build is one React and TypeScript single-page application assembled with Vite. The site layer owns routing, the top bar and the theme. A module page hosts the graph, tree or sorting visualizer. Each visualizer owns its data, algorithm execution and playback state, while shared components provide common layout and presentation utilities (Figure 1) [6].

This organization avoids making the shared shell responsible for algorithm logic. For example, the shell positions a Status panel, but the active algorithm determines its content. State is held in the browser; saved V10 layouts are versioned in local storage.

4.2 State ownership and reproducible playback

The graph foundation separates four concerns (Table 2). A persistent canvas is reused for editing and execution. Starting a run derives a sequence of typed steps (shown as frames) from the current document; stepping through that sequence derives the visible presentation. This reduces the risk of keeping a separate editing graph and animation graph that gradually disagree.

For example, moving vertex B should not change a shortest-path computation. Changing an edge weight should. Keeping position data separate from semantic input makes this distinction explicit. A run identity also protects against asynchronous playback events that arrive after the user stops or replaces a run.

Graph and tree algorithms build their complete step sequence when a run starts. The session then

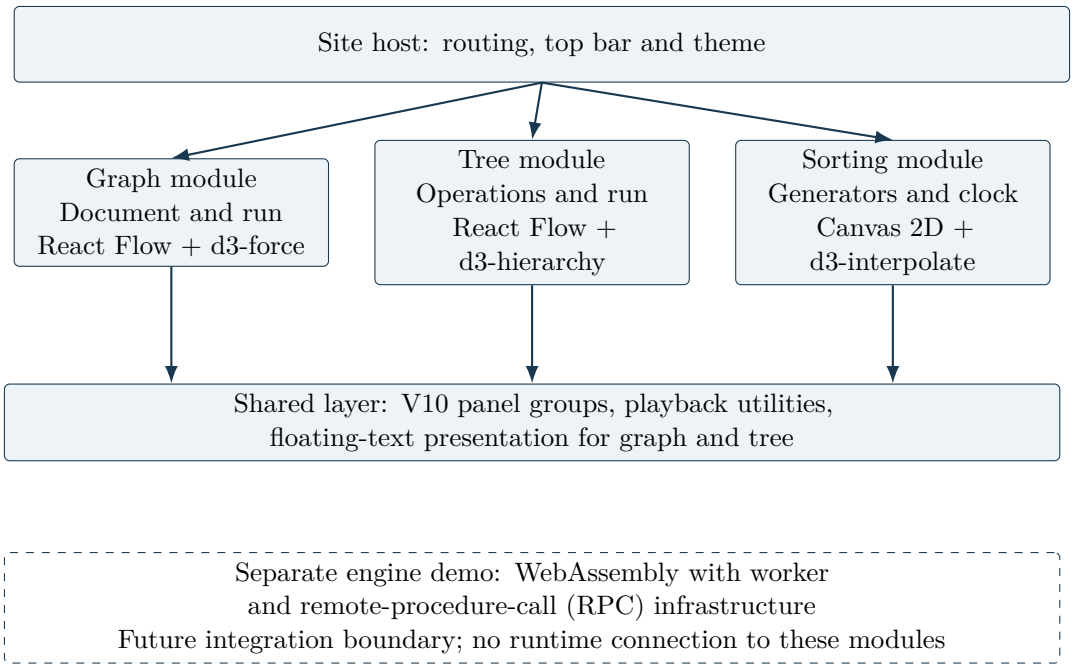


Figure 1: Current application boundaries. Each module renders its own drawing; the lower dashed box is a separate prototype, not a dependency of the visualization application.

Table 2: State separation in the graph visualizer.

State owner	Responsibility	Example consequence
Graph document	Vertices, edges, weights, stable identities and topology revision.	A semantic edit changes the topology revision, so the previous run is discarded.
Graph layout	Positions and pinned vertices.	Dragging a vertex changes the drawing without changing the algorithm input.
Run session	Step sequence, current index, run identity and playback status.	An outdated timer event cannot advance a newer run.
Presentation	Highlights, badges and derived edge/node styles.	Rewinding restores the presentation of the selected step.

moves between idle, paused, playing and finished states. Each selected step supplies consistent information for the drawing, pseudocode highlight and status text. This eager approach makes backward stepping and seeking straightforward, but increases startup time and memory use for long runs. These costs have not yet been benchmarked.

Tree operations add a distinction between a preview and a committed document. An insertion or deletion can be explored through intermediate states before its final result is applied. This lets the learner inspect an operation without treating every animation frame as a permanent edit.

4.3 Sorting uses a different execution model

Sorting produces operations from generators. A shared clock advances one, two or four panes, each with its own operation stream. The implementation retains executed history, buffers up to 2,048 future operations, and initially primes streams up to a 60,000-operation aggregate threshold before continuing lazily. Seeking backwards reconstructs the state from retained operations; it does not rely on stored checkpoints.

This approach avoids requiring every sort to finish generating before interaction begins. However, retained history is never trimmed (only the 240-value input cap bounds it), and a backward seek replays operations from the start to the target, so its cost grows with how late the target lies. Both are limitations for future measurement. A common operation index supports synchronized inspection, but operations from different algorithms are not equivalent units of computational cost.

4.4 V10: a shared panel-group layout

The V10 shell organizes supporting information around a fixed central canvas. The canvas cannot become a tab. Left, right and bottom regions contain groups of tabs; a user can move a panel into a group, split it off into a new group or place it in an empty region. The default arrangement puts Action and e-Lecture on the left, with Pseudocode and Status on the right (Figure 2).

The main engineering decision was to express layout changes in a pure model rather than embedding all rules inside pointer-event handlers. The model preserves panel membership, removes empty groups and ensures that each group's active tab belongs to that group. Required panels occur exactly once, optional panels at most once, and groups do not nest.

A common layout resolver is used for sizing, target validity and drag previews. A proposed move is first represented in the model and then resolved into geometry, so the preview and committed result use the same constraints. Expanded groups keep at least 240 pixels of width and 120 pixels of height. Each module sets its own drawing floor (360 by 320 pixels for the graph; 320 by 200 or 320

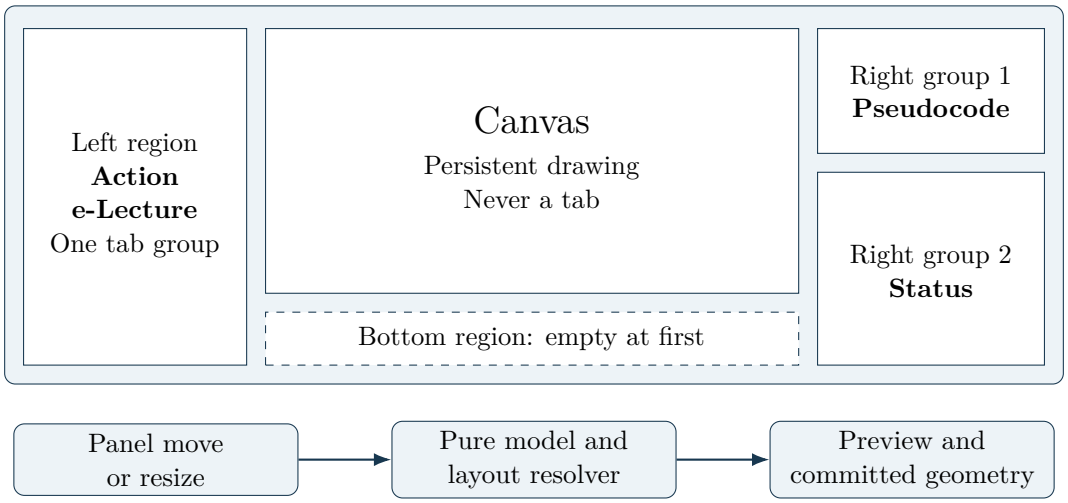


Figure 2: V10 layout and shared geometry calculation. The bottom region is shown as a target area; it does not reserve this height when empty.

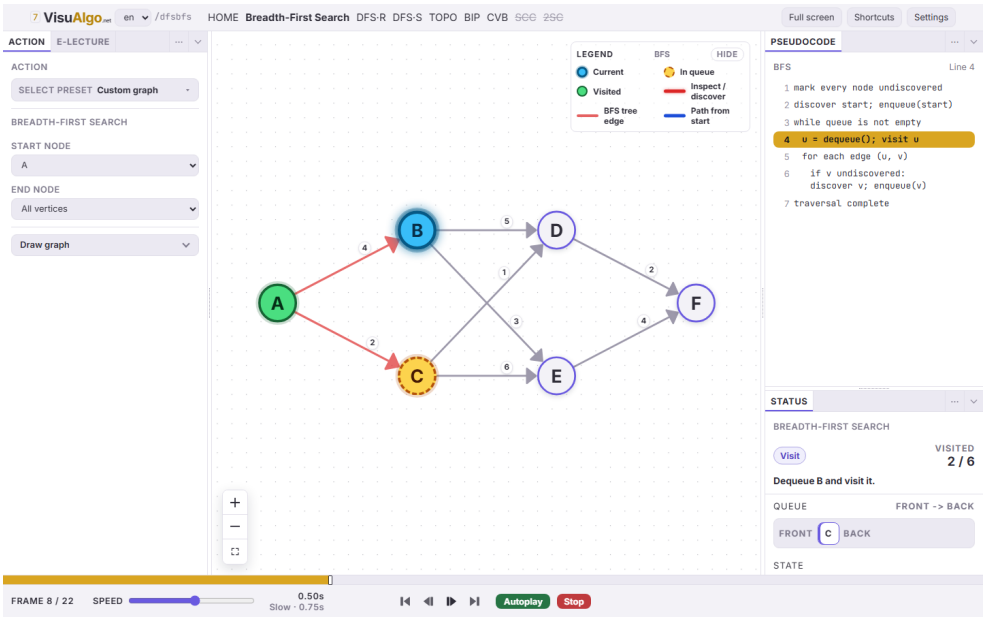


Figure 3: BFS at frame 8 of 22 in the default V10 arrangement. The drawing, pseudocode, Status panel and footer playback controls refer to the same selected step.

by 240 pixels for tree workspaces), which the resolver keeps when the viewport can accommodate it.

Saved layouts are versioned and repaired or replaced with defaults when invalid. Automated tests and manual checks were used to verify the layout behaviour: model tests run 4,000 seeded random operations across several viewport sizes with repeated serialization round trips; browser tests move tabs through the tab menu and drag a column splitter; and the graph and tree manual checklists cover tab drag, split and splitter resizing.

4.5 The shared shell in use

Figures 3 and 4 show why layout is part of algorithm interaction. A learner may need more space for pseudocode while tracing a run, then more space for input editing. A common shell permits this adjustment without changing the underlying algorithm state. The same design is used across the three visualizer families.

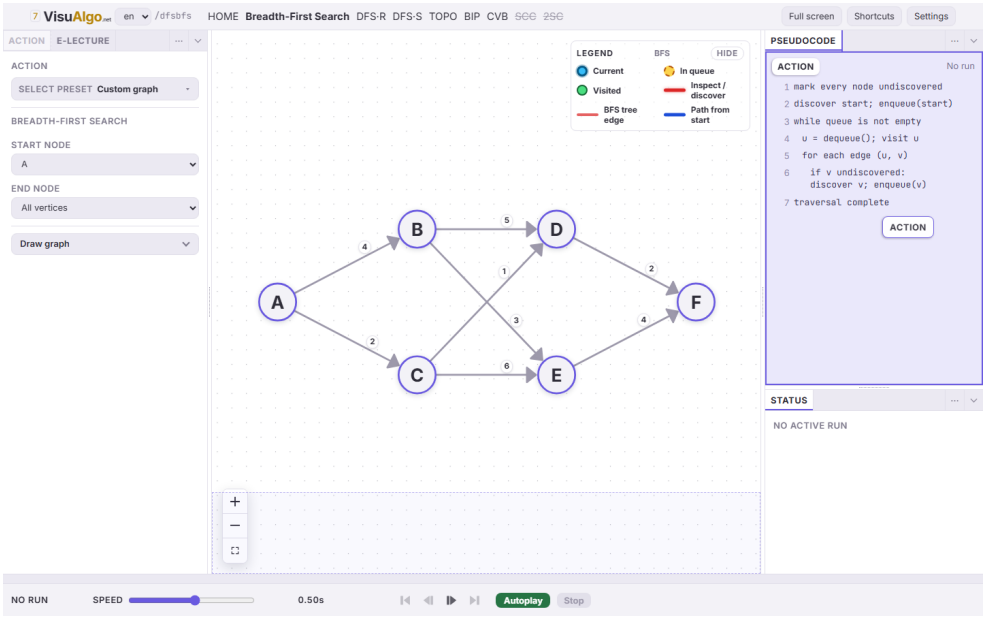


Figure 4: Dragging the Action tab onto the Pseudocode group shows the proposed drop area. This screenshot records the preview, not a completed drop.

4.6 Graph extensions and algorithm validation

The current graph module contains 11 algorithms, counting recursive and stack-based depth-first search (DFS) separately. Alongside breadth-first search (BFS), stack-based DFS, Dijkstra, Kruskal and Edmonds–Karp, the application supports recursive DFS, Bellman–Ford, Prim, topological sorting, bipartite checking, and cut-vertex/bridge detection. Topological sorting offers DFS and Kahn methods; bipartite checking offers DFS and BFS methods. BFS, Dijkstra and Bellman–Ford also accept an optional End node; their last frame then draws the shortest path to it.

These additions extend both computation and explanation. Recursive DFS exposes a different execution structure from a stack-based traversal. Topological sorting and bipartite checking provide witnesses when the input violates their conditions, rather than displaying only a final success or failure. Bellman–Ford and Prim were planned and built with Codex under my requirements and review.

A representative correctness issue: parallel edges. A low-link DFS must skip the parent *edge*, not every edge leading to the parent vertex. If two vertices are joined by two parallel edges, removing either edge leaves them connected. Ignoring both edges during DFS can incorrectly mark one as a bridge. Stable edge identities make the intended rule explicit; Figure 5 shows the detector's final result on one of VisuAlgo's example graphs.

For validation, the tests compare results with independently formulated oracles. Cut vertices and bridges are checked by removing a vertex or edge and recomputing connected components. Bellman–Ford is checked against Floyd–Warshall-based results, including negative-cycle reachability. Prim is compared with exhaustive forest candidates on small inputs. These oracles are algorithmically different from the implementations under test; this reduces shared assumptions, but is not a formal proof.

The recorded randomized cases include 700 random multigraphs for cut vertices and bridges (400 compared with the removal oracle and 300 checking the DFS-root rule), 140 signed graphs for Bellman–Ford, and 20 small graphs for Prim tested from every start vertex. Topological and bipartite methods have additional randomized checks. Fixed seeds make failures reproducible.

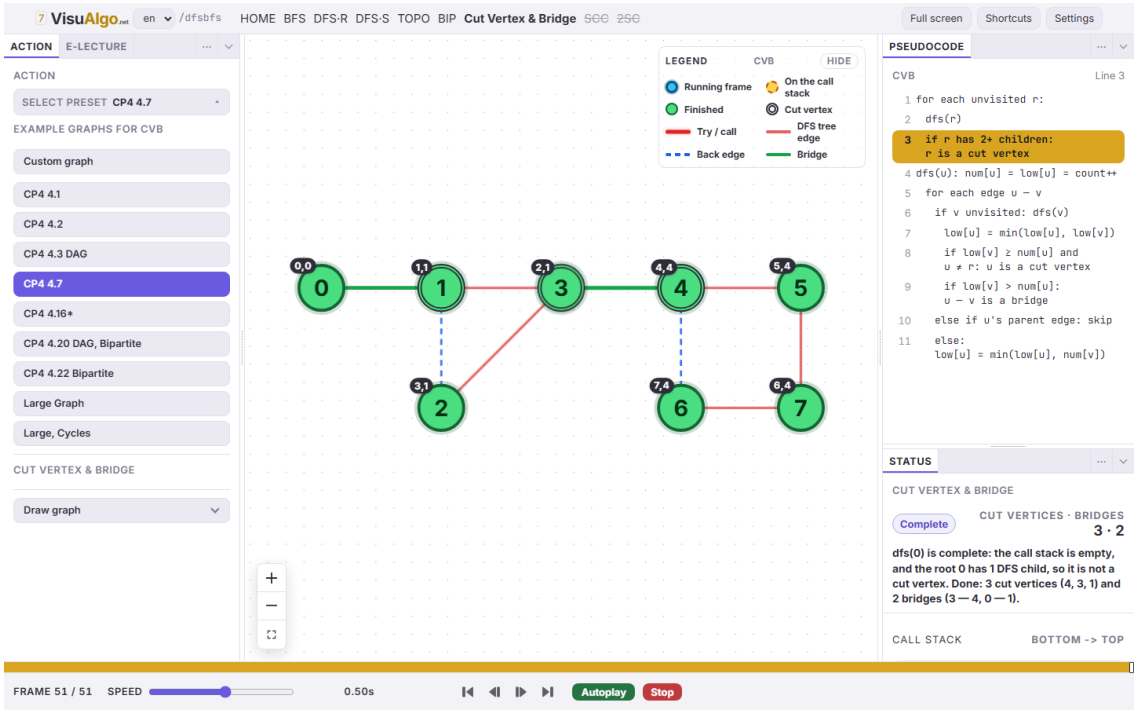


Figure 5: Final frame of cut-vertex and bridge detection on VisuAlgo's example graph labelled CP4 4.7: the result reports cut vertices 1, 3 and 4 and bridges 0–1 and 3–4.

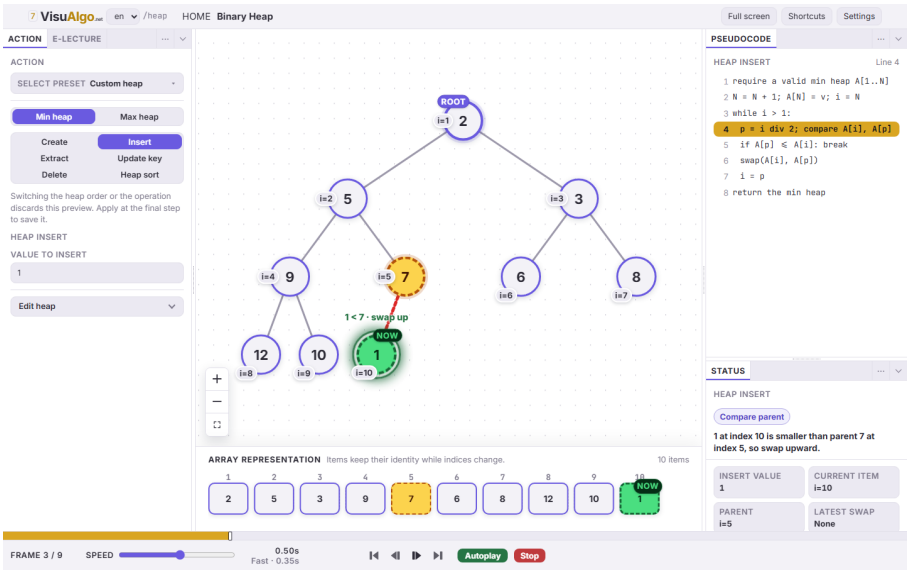


Figure 6: Heap insertion of 1, frame 3 of 9: the new item at index 10 is compared with its parent 7 at index 5. The floating-text note "$1 < 7 \cdot$ swap up" (Section 4.8) states the decision before the swap, and the array strip marks the same two items.

4.7 Tree operations and sorting comparison

The tree visualizer provides seven workspaces: lowest common ancestor (LCA), binary search tree (BST), AVL tree, binary heap, trie, segment tree and lazy segment tree. Work includes BST/AVL queries and traversals, AVL deletion, heap operations, and lazy range-add/range-sum behaviour. The heap links array and tree representations; intermediate changes remain previews until applied (Figure 6). These workspaces do not imply complete operation parity with VisuAlgo.

Sorting provides eight algorithms in a one-pane Study mode and two- or four-pane Compare mode. Panes share the input dataset and clock, making it possible to inspect different operation sequences

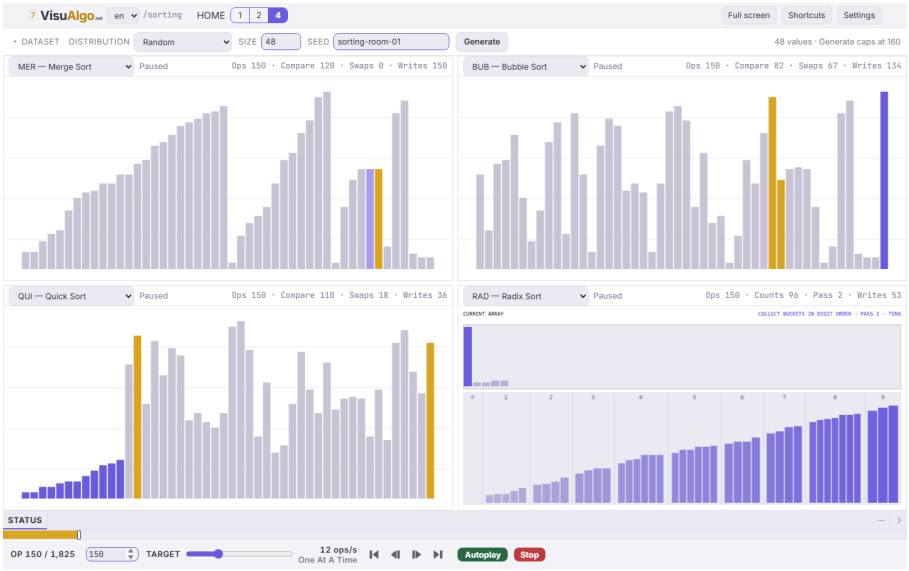


Figure 7: Four-pane comparison of Merge, Bubble, Quick and Radix sort on the same 48-element input. The shared operation index is a playback coordinate, not a measurement of relative runtime.

on the same data (Figure 7). Duplicate algorithm selection is prevented and randomized quicksort uses a reproducible seed.

4.8 Floating text: explanations on the canvas

A step can be meaningful even when the drawing hardly changes. For example, Dijkstra may reject a relaxation because the candidate distance is not shorter than the existing one. If the reason appears only in a separate Status panel, the learner must repeatedly look away from the relevant edge. After the Week 10 trial, I introduced floating text: short step notes placed on the canvas over the relevant node (for Dijkstra, the vertex that the relaxed edge reaches). It was one of several post-trial refinements (Section 6.1) and is intended to reduce how often users look at Status while following a run, keeping their attention on the canvas. Status remains available for persistent detail; reduced attention switching is a design aim, not a measured result.

Figure 8 shows an accepted update. Each relaxation writes one note over its target: green when the distance changes, as here, and red when it is kept, for example “already settled”. The implementation has a shared note description, placement calculation, timing logic and rendering layer. Graph and tree algorithms supply the semantic content; the shared layer controls how it is displayed. Current coverage includes Dijkstra, LCA, BST, AVL and Heap. Sorting, Trie, segment trees and the other graph algorithms do not yet use this layer.

Notes are rendered outside the zoom transform of the graph or tree canvas, so their text keeps a constant screen size while each note follows its node. Placement checks nearby obstacles, including node badges and edge-weight labels. It first tries preferred positions, then alternatives; when none is clear, it uses a minimum-overlap fallback. Dense graphs can therefore still produce overlap.

Timing adapts to playback. A note appears only when a step is reached by moving one step forward (Next or autoplay). After Next it lasts 1,600 milliseconds; during autoplay it is shortened to 90% of the step interval and omitted when less than 200 milliseconds would remain, so fast playback does not accumulate unreadable text.

Because viewing alone has rarely improved learning [3], the notes will be evaluated through active tasks (Table 4). The note layer is currently hidden from accessibility APIs, and the application

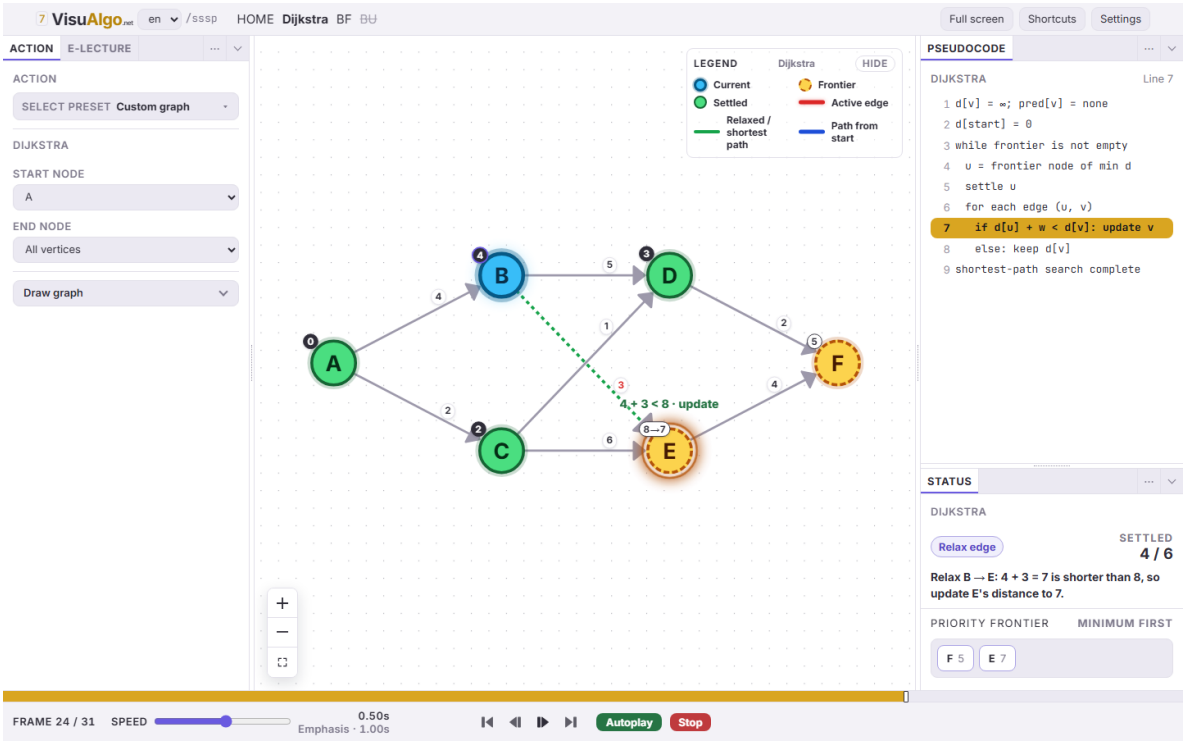


Figure 8: Dijkstra relaxation from B to E: the note explains that $4 + 3 < 8$, so E's distance changes from 8 to 7.

has not undergone a systematic accessibility evaluation. Future work should check whether the equivalent text in Status is sufficient and whether users can understand rejected actions, rapid transitions and dense cases without relying on colour or motion alone.

5 Current Results and Validation

The current build provides one site with runnable graph, tree and sorting modules (Figure 9). Of its 30 catalogue cards, 12 open a module and 18 are marked coming soon. Of the 26 cards transcribed from visualgo.net's module list, eight open: four offer every algorithm the original module lists (Sorting, BST/AVL, Binary Heap and Minimum Spanning Tree) and four offer some of them (Segment Tree, Graph Traversal, Single-Source Shortest Paths and Max Flow). The other four open cards are Trie and LCA, which visualgo.net has as separate pages, and this project's own Graph and Tree Editors. These labels count algorithms, not operations.

Table 3: Recorded checks on 1 November 2026 at application commit `662e7b3` [7].

Check	Result	Interpretation
Graph unit tests	654 passed	Algorithms, state and graph behaviour.
Tree unit tests	1,881 passed	Operations, state and presentation.
Sorting unit tests	571 passed	Operation streams and playback.
Shared / site unit tests	143 / 24 passed	Common components and site logic.
Total unit tests	**3,273 passed**	Recorded suite total.
Browser cases	151 passed	14 test files, run in Chromium.
Type check / build	Passed	Compile-time and production build checks.
Lint	0 errors	Two warnings remained.

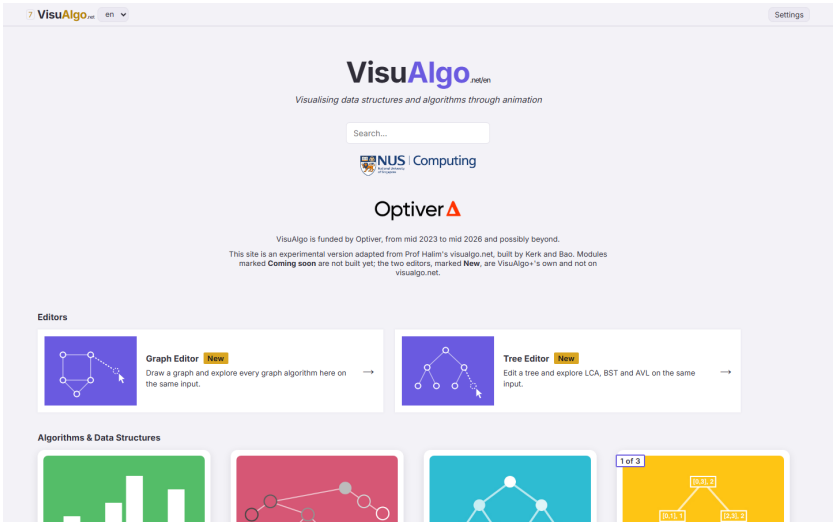


Figure 9: Top of the home page of the integrated application. Its credit line states that the site is adapted from visualgo.net; the module catalogue, including the coming-soon cards, continues below.

The recorded environment was Windows 11, Node 24.17.0, npm 11.13 and Playwright 1.64.1 with Chromium. The build emitted a large-chunk warning (about 705 kB before gzip and 206 kB after). From 4 September to 30 October, the repository's CI recorded 266 workflow runs, 105 of them in the visualizer workflows and the rest in Kerk's engine CI and the site deployment; eight failed, each followed by a passing run of the same workflow.

Passing tests support regression confidence within the tested cases (Table 3); they do not measure usability, accessibility, browser portability or learning. Automated tests and manual checks were used to verify interface behaviour (Section 4.4). The Week 10 trial provided formative feedback on usability (Section 6.1).

6 User Evaluation, Limitations and Next Steps

6.1 Week 10 formative user trial

In Week 10, Kerk and I invited several users, all NUS students, to use VisuAlgo+ with the V10 layout independently for 15–20 minutes. Participants varied in prior use of VisuAlgo, from frequent to infrequent users, and in algorithm experience, from experienced competitive programmers to beginners. We recorded each session on video, administered a short test afterwards and collected feedback. Kerk asked participants in advance for permission to record and use the videos, and all agreed; the recordings show only the screen and hand interactions, not faces.

The sessions yielded positive feedback as well as areas needing improvement. Reviewing the recordings helped us identify interactions that supported familiarization with the new interface and places needing refinement. This led to several UI/UX refinements: floating text, which keeps explanations close to the visualization (Section 4.8), and adjustments to the legend and Status panel.

6.2 Evaluation and implementation limits

Participant numbers, test questions, scores and detailed observations from the trial have not yet been compiled, so this report does not quote them. The trial provides formative design input; a post-use test alone does not establish learning gains or comparative usability, and the effect of floating text has not been measured. Performance benchmarks and a test-coverage percentage are also unavailable.

Table 4: Proposed tasks for the structured follow-up evaluation.

Task	Evidence to collect
Explain a Dijkstra relaxation	Correct explanation of an accepted and rejected update; use of notes, pseudocode and Status.
Trace heap insertion	Correct prediction of the next swap and understanding of preview versus Apply.
Rearrange the workspace	Successful tab move/split, recovery from an unwanted layout, errors and assistance needed.
Compare two sorting runs	Ability to explain an operation difference without treating operation index as elapsed runtime.

Several implementation limits are material. Graph and tree step sequences are built eagerly. Sorting keeps all history and replays it to seek backwards. Floating text covers only selected algorithms and may overlap in dense drawings. Browser validation is concentrated on desktop Chromium; mobile behaviour, cross-browser compatibility and accessibility remain incomplete. The manual developer checklists (20, 10 and 26 items for graph, sorting and tree) were run, but their runs were not recorded.

The separate engine is also a scope boundary. My unmerged experiment on Kerk's engine explored stable edge identity and a reusable browser client for worker/RPC lifecycle management. It helped identify integration requirements, but it is not part of the deployed visualizer path.

6.3 Proposed follow-up evaluation

I will first organize the Week 10 footage, test responses and feedback into anonymized observations linked to design changes, then conduct a more structured follow-up (Table 4), provisionally involving 6–10 students familiar with introductory data structures. The participant plan, consent procedure and any NUS ethics review requirement will be confirmed with the supervisor before recruitment. A small study can identify interaction problems; it cannot establish general learning effectiveness.

Each session will record task completion, time, errors, requests for help and short participant comments; any comparison with the original VisuAlgo will use matched tasks and inputs in counterbalanced order.

6.4 Proposed technical measurements

Measurements will use a fixed machine and production build with saved inputs of increasing size, covering run preparation, time to first visible step, seek latency, playback frame pacing and memory use. Each condition will be repeated at least five times, and the median reported with the spread and environment. These measurements should guide whether checkpointing, bounded history or a different execution boundary is necessary.

7 Plan for the Remaining Project

The following plan (Table 5) is proposed for discussion with the supervisor and Kerk. The SoC dates are the CA presentation in Reading Week (14–20 November 2026), the final report by 5 pm on Wednesday 7 April 2027, and the final presentation on 19–23 April 2027 (tentative); the other windows are proposals.

Table 5: Proposed milestones and completion evidence.

Window	Main work	Completion evidence
Nov–Dec 2026	CA presentation; consolidate Week 10 findings; stabilize the build; define follow-up tasks; keep automated and manual UI checks current.	Reproducible baseline, updated UI tests and checklists, and reviewed evaluation protocol.
January 2027	Investigate one engine integration slice with Kerk, starting with a simple graph algorithm.	Written input/step contract; matching results and playback semantics against the TypeScript reference.
February	Conduct follow-up task evaluation and technical measurements.	Anonymized observations, fixed benchmark inputs, environment and result tables.
March	Address high-impact findings; improve explanation and accessibility for selected modules; consolidate results and draft the final report.	Before/after regression checks, follow-up verification of the changed tasks, and a complete report draft.
April	Submit the final report by 7 April; final presentation and demonstration (19–23 April, tentative); e-copy by 10 May.	Reproducible release snapshot, final evaluation account and presentation materials.

7.1 Integration strategy and risks

Engine integration should begin with one vertical slice: graph input, engine execution, typed step conversion, rendering and backward/forward playback. The first milestone is behavioural agreement, including edge identities and cancellation, rather than replacing every TypeScript algorithm. If the engine contract or schedule is not ready, the existing implementation remains a functional baseline and the final report can document the integration study separately.

The main technical risks are the cost of long step sequences, inconsistent semantics between modules, and interaction failures that model tests do not expose. Measurements should precede a major playback redesign. Shared abstractions should be introduced only where behaviour is truly common; graph/tree eager sequences and sorting generators already demonstrate that identical controls need not imply identical execution internals.

The main project risk is scope growth. Adding every unfinished catalogue topic would reduce time for evaluation and refinement. I propose prioritizing the current graph, tree and sorting workflows, selected explanation gaps and a small engine experiment. Further features should be chosen with the supervisor according to educational value, missing coverage and available time.

7.2 Expected outcome

By the final assessment, the intended outcome is a documented and tested visualization foundation with a coherent interface, representative algorithm coverage, evidence from user tasks and technical measurements, and a clear account of engine integration progress. The first-semester work establishes the implementation base.

References

[1] D3 Project. Force simulation and tree layout documentation. `https://d3js.org/d3-force`; `https://d3js.org/d3-hierarchy/tree`. Accessed 3 November 2026.

[2] Steven Halim, Zi Chun Koh, Victor Bo Huai Loh, and Felix Halim. Learning algorithms with unified and interactive web-based visualization. *Olympiads in Informatics*, 6:53–68, 2012. `https://ioi.te.lv/oi/`

shtm/INFOL099.shtml.

[3] Christopher D. Hundhausen, Sarah A. Douglas, and John T. Stasko. A meta-study of algorithm visualization effectiveness. *Journal of Visual Languages and Computing*, 13(3):259–290, 2002. https://doi.org/10.1006/jvlc.2002.0237.

[4] OpenDSA Project. About OpenDSA. https://opendsa-server.cs.vt.edu/home/about. Accessed 4 November 2026.

[5] Clifford A. Shaffer, Matthew L. Cooper, Alexander Joel D. Alon, Monika Akbar, Michael Stewart, Sean Ponce, and Stephen H. Edwards. Algorithm visualization: The state of the field. *ACM Transactions on Computing Education*, 10(3):9:1–9:22, 2010. https://doi.org/10.1145/1821996.1821997.

[6] Truong Van Quoc Bao. VisuAlgo+ project records: Architecture and decisions, 2026. Working project record, 3 November 2026. Available on request.

[7] Truong Van Quoc Bao. VisuAlgo+ project records: Figures and validation, 2026. Working project record, 3 November 2026; recorded tests dated 1 November 2026. Available on request.

[8] Truong Van Quoc Bao. VisuAlgo+ project records: Scope, contributions and report mapping, 2026. Working project records, 3 November 2026. Available on request.

[9] xyflow. React Flow: Documentation. https://reactflow.dev/learn. Accessed 3 November 2026.

Project records [6–8] are the author's working records, not research publications. Screenshots show application commit `662e7b3`; diagrams were drawn for this report. VisuAlgo remains the source of the original educational material and examples.